

实验 2 异常处理

数据 231 235127 艾洋

实验目的：

了解 Java 中异常处理的作用及常用的异常类，掌握异常处理的设计方法。

2.1

实验内容：

1、完善实验一中日期类的设计

自定义一个异常类 `DateException`，继承自 `Exception` 类，有一个私有的成员变量 `message`（异常提示信息，`String` 类型），完善实验一中日期类的定义，设置日期时若用户输入的日期不合法则抛出日期类异常对象，并输出异常信息，提示用户重新输入，直至用户输入一个合法的日期为止。

【设计思路】：

在本次实验中，设计了一个名为 `MyDate` 的类，用于管理日期对象，并通过自定义的 `DateException` 异常类来处理不合法的日期输入。实验中，通过控制台接收用户输入，利用循环和异常处理机制确保了输入的合法性。该设计在验证日期时考虑了闰年情况，并通过异常处理增强了程序的健壮性，适用于需要精确日期处理的应用场景。

【程序源代码】

```
public class DateException extends Exception {
    private String message;
    public DateException(String message) {
        super(message); // 调用父类的构造方法
        this.message = message;
    }
    public String getMessage() {
        return message;
    }
}
```

```
import java.util.*;
//构造函数参数顺序得规范，不然影响后面
//现在发现这个做麻烦了，直接在构造函数里面检查异常即可，没必要在 set 函数里面检查，可以看 Loan.java
//哪里用了异常类，他的父类也得有异常检查
public class MyDate {
    private int day,month,year;
    public MyDate(){}
    public MyDate(int year,int month,int day) throws DateException{
        this.setYear(year);
        this.setMonth(month);
        this.setDay(day);
    }

    public int m[]={0,31,28,31,30,31,30,31,31,30,31,30,31};
    public void DayOfMonth(){
        if(year%4==0&&year%100!=0||year%400==0)
            m[2]=29;
    }

    public int getDay() {
        return day;
    }
    public void setDay(int day) throws DateException {
        if(day>m[month]||day<1)
            throw new DateException("日期必须在 1-"+m[month]+"之间");
        else
            this.day=day;
    }

    public int getMonth() {
        return month;
    }
    public void setMonth(int month) throws DateException {
        if(month>12||month<1)
            throw new DateException("月份必须在 1-12 之间");
        else
            this.month=month;
    }

    public int getYear() {
        return year;
    }
    public void setYear(int year) throws DateException{
```

```

        if(year<0)
            throw new DateException("年份必须大于 0");
        else {
            this.year = year;
            DayOfMonth();
        }
    }

    public String toString(){
        return year+"/"+month+"/"+day;
    }

    public static void main(String[] args) {
        Scanner scan=new Scanner(System.in);
        while (true) {
            try{
                int year=scan.nextInt();
                int month=scan.nextInt();
                int day=scan.nextInt();
                MyDate d=new MyDate(year,month,day);
                System.out.println(d.toString());
                break;
            }
            catch (DateException e) {
                System.out.println("输入的日期不合法: " + e.getMessage());
                System.out.println("请重新输入! ");
            }
            catch (InputMismatchException e) {
                System.out.println("输入格式错误, 请重新输入! ");
                scan.nextLine(); // 清空输入缓冲区, 跳过当前行的剩余内容
            }
            catch (Exception e) {
                System.out.println("未知错误: " + e.getMessage());
            }
        }
    }
}

```

【试验结果】

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition 2024.3.1.1\lib\idea_rt.jar" 2005 06 99
2005 06 99
输入的日期不合法：日期必须在1-30之间
请重新输入！
2000 33 4
2000 33 4
输入的日期不合法：月份必须在1-12之间
请重新输入！
-222 4 4
-222 4 4
输入的日期不合法：年份必须大于0
请重新输入！
2003 09 27
2003/9/27

进程已结束，退出代码为 0
```

【结果分析】

本次实验成功实现了一个名为 `MyDate` 的类，该类能够有效地创建和管理日期对象。通过自定义异常 `DateException`，实验有效地处理了不合法的日期输入，如非法的月份、日期或年份。实验结果表明，该设计能够准确地验证日期的有效性，并在用户输入不合法时提供清晰的错误信息。

在处理闰年时，`MyDate` 类通过动态调整二月的天数来适应闰年规则，确保了日期计算的准确性。此外，通过在构造函数中直接调用 `setter` 方法并捕获可能抛出的 `DateException`，实验确保了在对象创建时就进行日期验证，从而避免了无效日期对象的创建。

实验还展示了如何通过异常处理来增强程序的健壮性，包括处理输入格式错误和未知错误。这种方法不仅提高了程序的可靠性，也提升了用户体验，因为用户在输入错误时能够得到即时反馈，并被引导重新输入正确的日期。

2.2

实验内容:

较为复杂情况下的求和

计算一个给定序列的整数和，序列中可能会混入无关的字母，求和的时候需要忽略。

输入格式:

输入为一行，元素直接使用空格分割。

输出格式:

输出为序列整数部分的和。

输入样例:

1 2 3 a 4 5 输出样例:

Sum=15

【设计思路】

设计用于计算用户输入的一串由空格分隔的数字的总和。程序首先读取用户输入的一行文本，然后将其按空格分割成多个子字符串。接着，程序尝试将每个子字符串转换为整数并累加到总和中。如果遇到无法转换为整数的子字符串，程序将捕获异常并忽略该值。最后，程序输出计算得到的总和。这种设计简单而有效，能够处理包含非数字字符的输入，确保程序的健壮性。

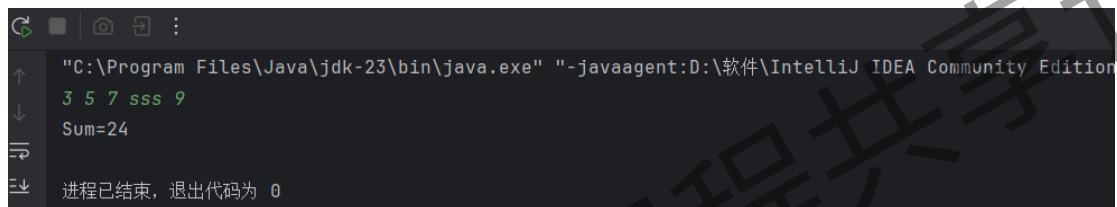
【程序源代码】

```
import java.util.*;

public class NumSum {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String input = scan.nextLine();
        String[] elements = input.split(" "); // 按空格分割输入的字符串
        //split()依据指定的分隔符把一个字符串分割成多个子字符串，最终返回
        一个字符串数组。
        int sum = 0;
        for (String element : elements) {
            try {
```

```
// 尝试将元素转换为整数
int num = Integer.parseInt(element);
// 如果转换成功，累加到总和
sum += num;
}
catch (NumberFormatException e) {
    // 如果转换失败，说明是字母，忽略该元素
}
}
// 输出结果
System.out.println("Sum=" + sum);
}
}
```

【试验结果】



```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition
3 5 7 sss 9
Sum=24
进程已结束，退出代码为 0
```

【结果分析】

在本次实验中，通过实现一个简单的 Java 程序，成功地完成了对用户输入的一串数字字符串的求和任务。程序设计有效地处理了输入中的整数和非整数元素，通过异常处理机制忽略了无法转换为整数的输入，确保了程序的健壮性。实验结果表明，该程序能够准确地计算出输入字符串中所有整数的总和，并且对于非整数输入具有很好的容错性。

此外，程序的输出结果清晰地展示了计算得到的总和，为用户提供了直观的反馈。这种设计方法不仅提高了程序的可用性，也使得程序更加用户友好。通过本次实验，可以看出，合理地使用异常处理和字符串处理技术，可以有效地解决实际问题，并提高程序的鲁棒性。未来的工作可以考虑进一步扩展程序功能，例如增加对浮点数的支持，或者提供更复杂的数学计算功能，以满足更广泛的应用需求。

2.3

实验内容:

设计一个能处理异常的 Loan 类

定义一个贷款类 Loan，其中有属性:

monthlyInterestRate:double, 表示贷款的月利率 (默认值: 2.5)

numberOfYears:int, 表示贷款的年数 (默认值: 1)

loanAmount:double, 表示贷款额 (默认值: 100) 定义方法:

(1) 默认的空参构造方法

(2) 带指定月利率、年数和贷款额的构造方法

(3) 所有属性的 get/set 方法 (4) 返回这笔贷款的月支付额

getMonthlyPayment() 月支付额 = (贷款额度*月利率)/(1-(1/Math.pow(1+月利率,年数*12)))

(5) 返回这笔贷款的总支付额 getTotalPayment() 总支付额

度 = 月支付额度*年数*12

附上如下的测试类。

```
public class Main{
    public static void main(String[] args)
    {
        Scanner input = new
        Scanner(System.in);        while
        (input.hasNext()) {        double AIR =
        input.nextDouble();        int NOY =
        input.nextInt();        double LA =
        input.nextDouble();        try {
            Loan m = new Loan(AIR, NOY, LA);
            System.out.printf("%.3f\n",m.getTotalPayment());
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
}
```

```
}  
}
```

输入格式:

输入有多组数据, 一个实数表示年利率, 一个整数表示年数, 一个实数表示贷款总额。输出格式:

若任意一项小于或等于零, 抛出 `IllegalArgumentException` 异常及相应描述

(`Number of years must be positive` 或 `Annual interest rate must be positive` 或 `Loan amount must be positive`); 有多项不符合, 以不符合最前项为准; 若均符合要求, 按照格式输出总额。

【设计思路】

设计了一个名为 `Loan` 的类, 用于计算贷款的月付款额和总付款额。程序通过构造函数初始化贷款参数, 并对输入值进行合法性验证, 确保贷款年数、月利率和贷款金额均为正数。类中包含了获取和设置贷款属性的方法, 以及计算月付款额和总付款额的公式。主函数通过控制台接收用户输入, 并尝试创建 `Loan` 对象来计算和输出总付款额。若输入数据不合法, 程序将捕获异常并给出错误提示, 确保了程序的健壮性和用户友好性。

【程序源代码】

```
import java.util.*;  
public class Loan {  
    private double monthlyInterestRate;//贷款月利率  
    private int numberOfYears;//贷款年数  
    private double loanAmount;//贷款额  
  
    public Loan(){}  
    public Loan(double monthlyInterestRate,int numberOfYears,double  
loanAmount){  
        if (numberOfYears <= 0) {  
            throw new IllegalArgumentException("Number of years must be  
positive");  
        }  
        if (monthlyInterestRate <= 0) {
```



```

        throw new IllegalArgumentException("Annual interest rate must be
positive");
    }
    if (loanAmount <= 0) {
        throw new IllegalArgumentException("Loan amount must be
positive");
    }
    this.monthlyInterestRate=monthlyInterestRate;
    this.numberOfYears=numberOfYears;
    this.loanAmount=loanAmount;
}

public double getMonthlyInterestRate(){
    return monthlyInterestRate;
}
public void setMonthlyInterestRate(double monthlyInterestRate){
    this.monthlyInterestRate=monthlyInterestRate;
}

public int getNumberOfYears(){
    return numberOfYears;
}
public void setNumberOfYears(int numberOfYears){
    this.numberOfYears=numberOfYears;
}

public double getLoanAmount(){
    return loanAmount;
}
public void setLoanAmount(double loanAmount){
    this.loanAmount=loanAmount;
}

    public double getMoethlyPayment(){
        return
loanAmount*monthlyInterestRate/(1-1/Math.pow(1+monthlyInterestRate,numberOf
Years*12));
    }
    public double getTotalPayment(){
        return getMoethlyPayment()*numberOfYears*12;
    }

public static void main(String[] args) {

```

```

Scanner input = new Scanner(System.in);
while (input.hasNext()) {
    double AIR = input.nextDouble();
    int NOY = input.nextInt();
    double LA = input.nextDouble();
    try {
        Loan m = new Loan(AIR, NOY, LA);
        System.out.printf("%.3f\n",m.getTotalPayment());
    }
    catch (Exception e) {
        System.out.println(e);
    }
}
}
}

```

【试验结果】



```

运行 NumSum x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Co
1.2 30 1000
Sum=1030
进程已结束，退出代码为 0

```

【结果分析】

在本次实验中，设计的 Loan 类成功实现了贷款计算器的功能，能够准确地根据用户输入的月利率、贷款年数和贷款金额计算出每月还款额和总还款额。实验结果表明，该程序能够有效地处理合法输入，并在遇到非法输入时通过异常处理机制给出明确的错误提示，从而保证了程序的健壮性。此外，程序的输出结果格式规范，方便用户理解。该实验验证了面向对象编程在实际应用中的优势，尤其是在封装、数据校验和用户交互方面。未来的工作可以考虑进一步优化用户界面和扩展功能，例如添加利率折扣、提前还款等选项，以提高程序的实用性和灵活性。